

第一节 单元测试

1. `isdigit()`，判断字符串是否全为数字。

例如：

```
A = "123456"

B = "123qwe"

print(A.isdigit())    # 返回结果为 True
print(B.isdigit())    # 返回结果为 False
```

2. `len()`，获取对象（字符、列表、元组等）长度或项目个数。

例如：

```
A = "123456"

print(len(A))    # 返回的结果为 6
```

3. `enumerate()`，将一个可遍历的数据对象(如列表、元组或字符串)组合为一个索引序列，同时将索引值和数据列出来。

例如：

```
A = ["apple", "banana", "watermelon"]

for i, j in enumerate(A):

    print("索引值: ", i) # 返回列表 A 的索引值，如： 0
    print("元素值: ", j) # 返回索引值对应的元素值，
    如： apple
```

第二节 嵌套循环

1. 嵌套循环：将一个循环放进另一个循环中，分为外循环和内循环

环两部分。

2. 循环方式：外循环**每迭代一次**，内循环完成**所有迭代**。

例如：

```
for x in range(5,8):  
    for i in range(1,6):  
        print(i,"x",x,"=",i * x)  
    print()
```

内循环打印一个乘法表 外循环分别用值 5,6,7去迭代一次

第三节 可变循环

1. 在一个 for 循环的 range() 函数中会使用常数，程序运行时循环总会运行相同的次数，我们称为**硬编码**，就是将数据直接写入到代码中。

2. **可变循环**：用户自己决定循环次数，定义**变量**来控制。

例如：

控制循环次数，画出指定数量的圆。

```
from turtle import *  
  
A = int(input("你想画几个圆？"))  
  
for i in range(0,A):  
    circle(100)  
    lt(360/A)
```

3. **阶乘**：一个**正整数**的阶乘是所有**小于及等于**该数的正整数的积，并且**0 的阶乘为 1**。表示方式为： $n!=1*2*3*4\ldots*(n-1)*n$ 。

例如：

$$1! = 1*1$$

$$2! = 1*2$$

$$3! = 1*2*3$$

4. python 求阶乘：

```
n = int(input("请输入一个正整数："))
c = 1
if n>=1:
    for i in range(1,n+1):
        c = c*i
    print("%s 的阶乘为： %s"%(n,c))
else:
    print("请输入一个正整数！！！")
```

第四节 函数积木

1. **函数**：组织好的，可**重复使用**的，用来实现单一，或相关关联功能的**代码段**。就像积木一样，不同积木有**不同的作用**，可以**拼接组合**在一起，完成一个复杂的任务。**函数**能提高代码的**模块性**和代码的**重复利用率**。

2. **函数种类**：分为**内建函数**和**自建函数**。

3. **内建函数**：Python 解释器内置的许多始终可用的功能和类型。

例如：

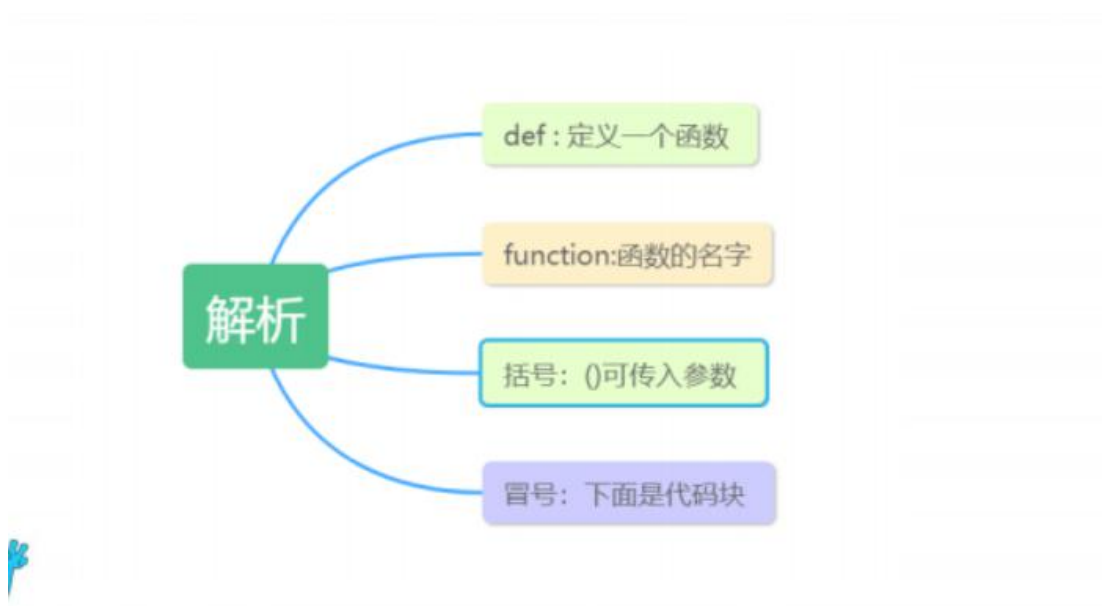
		内建功能		
abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id()	object()	sorted()
ascii()	enumerate()	input()	oct()	staticmethod()
bin()	eval()	int()	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set()	

参考链接：

https://blog.csdn.net/qq_23869697/article/details/83894174

4. 自建函数：具有一定功能的自定义函数。

(1) 定义函数：def function(arg1, arg2...):



例如：

from turtle import *

```
def draw_circle():  
    for i in range(360):  
        fd(1)  
        lt(1)
```

- (2) **调用函数**：目的是为了**运行**函数里的代码，实现函数的功能，需要用到**函数名和括号**，有时括号里会传入参数。创建了函数，如果不进行调用，那么该函数永远都不会执行。

例如：

```
draw_circle() # 调用上面创建的函数，画一个圆。
```

第五节 传递参数

1. **参数**：也叫参变量，是一个**变量**。
2. 参数类型：
 - (1) **位置参数**：调用函数时根据函数定义的**参数位置**来传递参数。

例如：

```
def addition(a,b):  
    print(a+b)  
  
addition(1,2) # 打印的结果为 3
```

- (2) **默认参数**：为参数提供默认值，默认参数要**放在位置参数后面**。

例如：

```
def addition(a, b, c=5):
```

```
    print(a+b+c)
```

```
addition(1, 2) # 打印的结果为 8
```

(3) **可变参数**：可变参数**个数是可变的**，可以是 0 个，也可以是多个，**不限制参数数量**。表示形式为在对应参数前加上“*”。调用函数时，所有参数会自动组成一个**元组**。

例如：

```
def calc(*numbers):
```

```
    sum = 0
```

```
    for i in numbers:
```

```
        sum = sum + i
```

```
    print(sum)
```

```
calc(1, 2, 3) # 打印的结果为 1, 3, 6
```

(4) **关键字参数**：允许你传入 0 个或任意多个**含参数名的参数**，这些关键字参数在函数内部自动组装为一个**字典**。表示形式为在对应参数前加上“**”。

例如：

```
def person(name, age, **kw):
    print("name: %s\nage: %s"%(name, age))
    if len(kw)>0:
        for i, j in kw.items():
            print("%s: %s"%(i, j))

person("Sunrise", 18, favorite="listen", place="shanghai")
```

3. **return**: 函数调用的时候计算结果通常是通过返回值带出的。**return** 则将它后面的部分作为返回值作为函数的输出，可以用变量来接收。

注意:**return** 语句返回的结果不能直接输出到控制台。

例如:

```
def addition(a,b):  
    return a+b  
  
sum = addition(1,2) # 变量接收返回值  
print(sum) # 打印的结果为 3
```

第六节 函数升级

1. **递归函数**: 一个函数在内部调用自身。

例如:

```
def f(n):  
    if n==1:  
        return 1  
    else:  
        return n*f(n-1)  
  
A = f(1) # 接收返回值  
B = f(2)  
C = f(5)  
  
print(A) # 打印的结果为 1  
print(B) # 打印的结果为 2
```

print(C) # 打印的结果为 **120**

计算步骤为：

```
f(5)
5*f(4)
5*(4*f(3))
5*(4*(3*f(2)))
5*(4*(3*(2*f(1))))
5*(4*(3*(2*1)))
5*(4*(3*2*1))
5*(4*3*2)
5*(4*6)
5*24
120
```

如果计算f(5),根据函数定义,计算过程就是左边这样的。

2. **高阶函数**：把函数**作为参数**传入**另一个函数**。

(1) 调用函数将结果赋值给变量。

例如：

```
A = abs(-10)
```

```
print(A) # 打印的结果为 10
```

(2) 直接将函数本身赋值给变量。

例如：

```
A = abs
```

```
print(a(-10)) # 打印的结果为 10
```

(3) 综上。

例如：

```
def addition(x, y, a):
```



```
return a(x)+a(y)
```

```
sum = addition(-8,-10,abs) # 接收返回值
```

```
print(sum) # 打印的结果为 18
```

3. `map()`, python 内置的高阶函数, 可以接受两个参数, 得到一个 `map` 对象, 第一个参数是函数, 第二个参数通常是列表 `list`, 执行方式为把函数依次作用在列表的每个元素上, 返回的结果是一个新的列表。

例如:

```
list1 = [1,2,3]
```

```
def function(x,y=9):
```

```
    return "%s*%s=%s"%(x,y,x*y)
```

```
list2 = map(function,list1) # 接收返回值
```

```
print(list2)
```

```
# 打印新的列表 ["1*9=9", "2*9=18", "3*9=27"]
```

4. 变量作用域: 指变量作用范围(区域), 分为全局变量和局部变量。

(1) 全局变量: 一般指在函数外部定义的变量。

例如:

```
num = 100 # 全局变量会影响到函数内的变量
```

```
def func():
```

```
    X = num + 100
```

```
    print(X)
```

`func()` # 结果为 200

(2) 局部变量：指在函数内部定义的变量。

例如：

```
num = 100
```

```
def func():
```

```
    # 函数内的计算，默认使用局部变量
```

```
    num = 200
```

```
    X = num + 100
```

```
    print(X)
```

`func()` # 结果为 300

(3) `global`，在函数中将某个局部变量定义为全局变量时，可以使用关键字 `global`。

例如：

```
num = 100 # 全局变量，在函数外部
```

```
def func():
```

```
    global num # 将局部变量定义为全局变量
```

```
    num = 200 # 局部变量，在函数内部。此时该
```

变量发生变化，全局变量也会变化。

```
    print(num)
```

`func()` # 结果为 200

```
print(num) # 全局变量发生变化，结果为 200
```

第七节 综合运用

1. 函数可将复杂的问题**简单化**，**结构化**，**逻辑化**。
2. 函数将程序分为不同的块，每个块有自己的功能，可被**反复调用**。

第八节 调用模块

1. **模块(Module)**：为了编写可维护的代码，会将函数分组，放到一个 Python 文件里，在 python 中以 **.py** 结尾的文件就是一个模块。函数相当于单个积木块，模块相当于一盒积木块，具有众多功能。

2. **模块的作用**：

- (1) 提高了代码的**可维护性**。
- (2) 模块可以被**重复使用**。
- (3) 避免函数名和变量名冲突。

3. **调用模块**：使用 **import** 进行模块的使用。

例如：

```
import turtle # 导入 turtle 模块
```

4. **math** 模块：处理**数学问题**的模块。

- (1) **ceil(a)** # 对 a 向上取整
- (2) **floor(a)** # 对 a 向下取整
- (3) **pow(a, b)** # a 的 b 次幂
- (4) **sqrt(a)** # a 的算术平方根
- (5) **sin(a)** # a 的正弦值

(6) `cos(a)` # `a` 的余弦值

5. **完全平方数**：如果一个**正整数** `a` 是某一个**整数** `b` 的平方，那么这个**正整数** `a` 叫做**完全平方数**。

6. **算术平方根**：若一个**非负数** `x` 的平方等于 `a`，即 $x^2=a$ ，则这个数 `x` 叫做 `a` 的**算术平方根**。

第九节 时间模块

1. 时间的表示方法：

(1) **格式化字符串**：

`strftime("%Y-%m-%d %H:%M:%S")`，获取**当前时间**，以年月日时分秒的格式表示出来。

例如：

```
import time

now = time.strftime("%Y-%m-%d %H:%M:%S")

print(now)

# 显示的结果为 2019-10-11 15:32:47

# 以计算机当前系统时间为主
```

(2) **时间戳**，指格林威治时间 1970 年 01 月 01 日 00 时 00 分 00 秒（北京时间 1970 年 01 月 01 日 08 时 00 分 00 秒）起至今的总秒数。

`time()`，获取**当前**的时间戳，获取的结果是一串**数字**。

例如：

```
import time

now = time.time()

print(now)

# 显示的结果为 1570779291.86

# 以计算机当前系统时间为主
```

(3) `struct_time` 结构体时间(元组)。

`localtime()`，获取结构体时间，以元组的形式表示出来。

例如：

```
import time

now = time.localtime()

print(now)

# 显示的结果为

time.struct_time(tm_year=2019, tm_mon=10,
tm_mday=11, tm_hour=15, tm_min=35, tm_sec=19,
tm_wday=4, tm_yday=284, tm_isdst=0)

# 以计算机当前系统时间为主

对应意义：
```

0	tm_year (年)	比如2011
1	tm_mon (月)	1 - 12
2	tm_mday (日)	1 - 31
3	tm_hour (时)	0 - 23
4	tm_min (分)	0 - 59
5	tm_sec (秒)	0 - 61
6	tm_wday (weekday)	0 - 6 (0表示周一)
7	tm_yday (一年中的第几天)	1 - 366
8	tm_isdst (是否是夏令时)	默认为-1

(4) 时间格式转换:

- ① **strptime()**, 格式化字符串时间转换为结构化时间, 反之, 使用 **strftime()**。

例如:

```
import time

Time = "2019-10-11 15:41:36"

St_time = time.strptime(Time,
"%Y-%m-%d %H:%M:%S")

print(St_time)

Sf_time =time.strftime(
"%Y-%m-%d %H:%M:%S", St_time)
```

- ② **gmtime()** 或者 **localtime()**, 时间戳时间转换为结构化时间, 反之, 使用 **mktime()**。

例如:

```
import time

Time = 1570779291.86
```

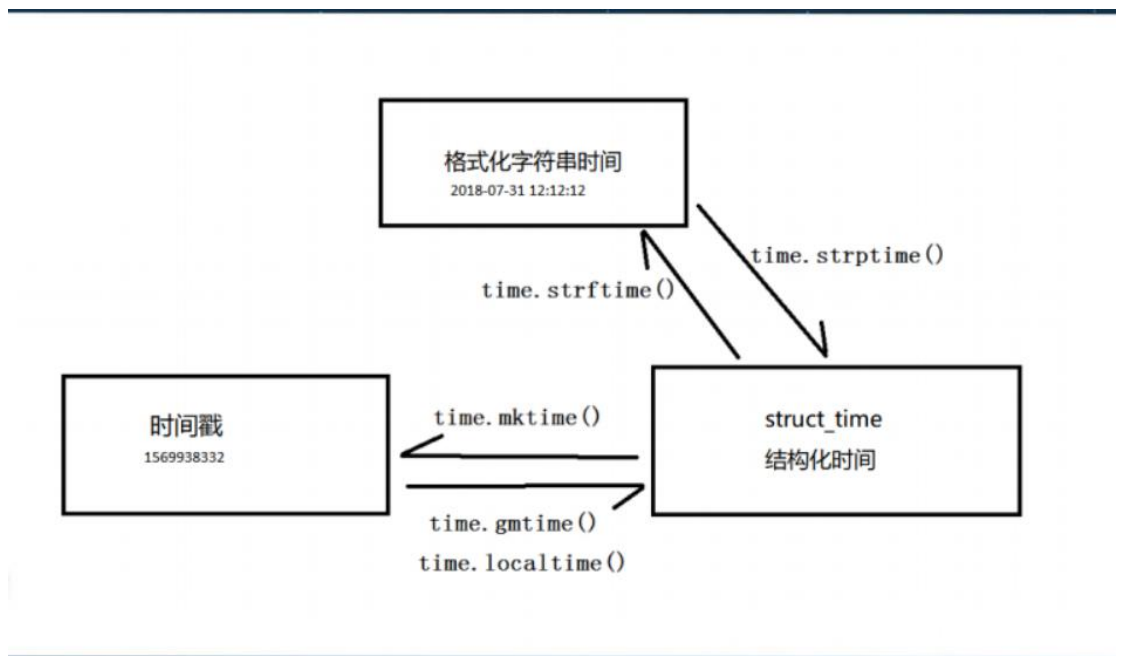
```
St_time = time.gmtime(Time)

print(St_time)

Mk_time = time.mktime(St_time)

print(Mk_time)
```

- ③ 时间戳时间转换为格式化字符串时间，需要先转换为对应的结构化时间，再进行相应的转换。反之亦然。



2. `datetime.datetime.now()`，获取当前时间。

例如：

```
import datetime
import time
start_time = datetime.datetime.now()
print("程序睡觉中....")
time.sleep(5) # 程序休眠 5 秒
print("程序苏醒了....")
end_time = datetime.datetime.now()
print("程序睡觉时间:%s 秒"%(end_time-start_time).seconds)
# 该程序旨在测试程序运行时间
```

第十节 随机模块

1. **random** 模块：用来生成随机数，

使用方式：**import random**

2. **randint(x, y)**：随机生成一个介于 x-y 之间的整数。

例如：

```
import random  
  
a = random.randint(1, 100)  
  
print(a)
```

3. **random()**：随机生成一个介于 0-1 之间的数，**包含 0 但不包含 1**。如果想得到其他范围的数，乘以某个数即可。

例如：

```
import random  
  
a = random.random()*10  
  
print(a)
```

4. **choice()**：从列表、元组、字符串中随机选取一个元素。

例如：

```
import random  
  
a = "Hello World!"  
  
b = random.choice(a)  
  
print(b)
```

5. **sample()**：从列表、元组、字符串中随机选取 **n** 个元素，

返回有 **n** 个元素的**列表**。

例如：

```
import random

number = [1, 2, 3, 4, 6, 8, 9]

# 第一个参数是列表、元组或字符串，第二个参数是选取的元素个数。

number_list = random.sample(number, 3)

print(a)
```

6. `turtle.window_width()`:在海龟画图中可以获取画布的宽度。

例如：

```
import turtle

a = turtle.window_width()

print(a)
```

7. `turtle.window_height()`:在海龟画图中可以获取画布的宽度。

例如：

```
import turtle

a = turtle.window_height()

print(a)
```

第十一节 综合运用

自我总结，自我实践，自我勉励，自我提升。

第十二节 类的实例

1. **编程范式**：又可称编程范型、程序设计法，是一类典型的**编程风格**。比如：**面向过程编程**和**面向对象编程**，这就是两种编程风格。
2. **面向过程编程**：就是把一个大问题分解成一个个小的步骤，通过完成一个个小的步骤来完成一个程序。
3. **面向对象编程**：就是模拟现实生活中的某些现象，用类和对象来描述这个世界。
4. **类**：即类别，种类。只要被称为一类的物体，他们都有着相似的特征和行为方式，因此类就是有**共同特征**和**行为**的事物的集合，比如：人类，具有耳朵眼睛嘴巴，会吃喝跑跳。
5. **对象**：就是某个物体，是一类事物中具体的一个。比如：**人类中的黄种人**。
6. **类的创建**：python 中使用 **class** 创建一个类，后面写上类名，类名**首字母大写**，采用驼峰命名法（骆驼）。

例如：

```
class Person():
```

```
    gender = "man"
```

```
# Person 即类名，class 是关键字。
```

7. **创建对象**：赋值号左边创建一个变量，右边写上类的名称

和括号，得到一个具体的对象，称它为类的实例。也可以说对象是类的实例化。

例如：

```
class Person():  
    gender = "man"  
person1 = Person()  
person2 = Person()
```

8. 对象的特征称之为属性，动作或行为称之为方法。

9. 定义属性：对象.属性名 = 属性值。

例如：

```
class Person():  
    gender = "man"  
person1 = Person()  
person2 = Person()  
person1.name = "sunrise"  
print("姓名： %s"%person1.name)
```

10. 定义方法：方法就是能够做什么，它是一些函数，可以调用方法完成某些事情。

例如：

```
class Person():  
    gender = "man"  
    def run():
```

```
print("奔跑")

person1 = Person()

person2 = Person()

person1.name = "sunrise"

print("姓名: %s"%person1.name)

person1.run()
```

第十三节 魔法方法

1. `__init__()`: 构造函数, 创建这个类的对象时, 就会自动运行该方法, 其为所有对象共用的, 就像一个模板一样。
2. 定义 `__init__()`: `init` 两边各有两条下划线, 括号内可传入参数, 第一个是 `self`, 表示创建类的实例本身。

例如:

```
class Person():

    def __init__(self, name, age):

        # self.name 中的 name, 不是必须的,
        # 可以是其他变量。

        self.name = name

        self.age = age
```

3. 给属性指定默认值:

例如:

```
class Person():
```

```
def __init__(self, name):  
    # self.name 中的 name, 不是必须的,  
    可以是其他变量。  
    self.name = name  
    self.age = 18
```

默认属性不需要再__init__方法中设置形参

4. 修改属性的值:

(1) 直接修改属性的值。

例如:

```
class Person():  
    def __init__(self, name):  
        # self.name 中的 name, 不是必须的,  
        可以是其他变量。  
        self.name = name  
        self.age = 18  
  
person1 = Person("sunrise")  
person1.age = "20"
```

(2) 用方法修改属性的值。

```
class Person():  
    def __init__(self, name):  
        # self.name 中的 name, 不是必须的,  
        可以是其他变量。
```

```
        self.name = name

        self.age = 18

    def update(self, a):

        self.age = a

person1 = Person("sunrise")

person1.update("20")
```

(3) 用方法对属性的值递增。

例如：

```
class Person():

    def __init__(self, name):

        # self.name 中的 name, 不是必须的,
        # 可以是其他变量。

        self.name = name

        self.age = 18

    def add(self, b):

        self.age += b

person1 = Person("sunrise")

person1.add(2)
```

第十四节 继承多态

1. 面向对象编程特点：封装、继承、多态。
2. 继承：一个类从其他类那里继承属性和方法，每个类共享相同的属性和方法，该类称之为子类，被继承的类为

父类。

例如：

```
class Dog():  
    def eat(self):  
        print("吃")  
  
class Too(Dog):  
    pass  
  
class Loo(Dog):  
    Pass  
  
a = Too()  
b = Loo()  
  
a. eat()  
b. eat()
```

如上，Dog 类是被继承的类，它是父类，Too 和 Loo 是子类，可以从父类继承一些属性和方法。

3. 父类定义属性后，根据子类创建对象时也需要提供一个实参。

例如：

```
class Dog():  
    def __init__(self, name):  
        self.name=name  
  
class Too(Dog):
```

```
pass
```

```
a = Too("大黄")
```

4. 绑定父类的构造函数：如果要给子类增加属性，应当显示调用父类的构造函数。

例如：

```
class Dog():  
    def __init__(self, name):  
        self.name=name  
  
class Too(Dog):  
    def __init__(self, name, hair):  
        Dog.__init__(self, name)  
        Self.hair=hair  
  
a = Too("大黄", "白色")
```

5. **多态**：对于同一个事物，可以有同名的两个或多个方法，用一种调用方式，就能得到不同的结果。

例如：


```
class Triangle:
    def __init__(self,width,height):
        self.width = width
        self.height = height
    def getArea(self):
        area = self.width*self.height / 2.0
        print(area)
class Square:
    def __init__(self,size):
        self.size = size
    def getArea(self):
        area = self.size*self.size
        print(area)
myTriangle = Triangle(4,5)
mySquare = Square(10)
myTriangle.getArea()
mySquare.getArea()
```

在计算两个形状的面积时，都采用了getArea ()这个方法。每个形状中，这个方法做的工作不同，这就是一个多态的例子。

第十五节 多样继承

1. 面向对象三大特点：

面向对象三大特性

封装：将内容封装到某个地方，形成模板，可反复调用

继承：子类可获得父类的属性和方法，减少代码的重用

多态：用一种方法名，多个对象调用它，获得不同结果

2. 继承分为两种：

(1) **单继承**：只有一个父类。

(2) **多继承**：会有多个父类。

例如：

```
class A():
    def test1(self):
        print("我是父类A")
class B():
    def test2(self):
        print("我是父类B")
class C(A,B):
    def test3(self):
        print("我是子类C")
c=C()
c.test1()
c.test2()
c.test3()
```

3. **私有属性**：如果在属性名前面加上两个下划线“__”，则表明该属性为私有属性，否则为公有属性。

例如：

```
def __init__(self, name)
```

```
self.__name = name
```

4. **私有方法**：不能直接调用，必须构造另一个函数来调用，它的作用就是在开发过程中保护核心代码。

例如：

```
class Money():
    def __get_money(self):
        print("可以取款")
    def person(self,name,password):
        if name == "名字"and password == "123456":
            self.__get_money()
        else:
            print("输入的名字或密码有误")
w = Money()
w.person("名字","123456")
```

第十六节 单元测试

自我总结，自我实践，自我勉励，自我提升。

